

LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference

Yihua Cheng^{1*} Yuhan Liu^{1,2*} Jiayi Yao^{1,2*}
Yuwei An¹ Xiaokun Chen¹ Shaoting Feng¹ Yuyang Huang^{1,2} Samuel Shen¹
Kuntai Du¹ Junchen Jiang^{1,2}

Abstract

今天的大型语言模型（LLM）推理系统为了简化处理，独立对待各个引擎和查询，但这导致了显著的资源效率低下。尽管已有方案提出通过在查询间复用键值（KV）缓存来避免冗余计算，并通过将单个查询拆分到不同引擎以提高 GPU 利用率，但若无法高效地在 LLM 推理引擎和查询之间进行 KV 缓存的卸载与通信，这些方案的潜力便无法实现。我们提出了 **LMCache**，这是首个且迄今为止最高效的开源 KV 缓存解决方案，能够提取并存储现代 LLM 引擎（如 vLLM 和 SGLang）生成的 KV 缓存，并在引擎与查询之间共享这些缓存。LMCache 在 LLM 引擎接口中暴露 KV 缓存，有效将 LLM 引擎从独立的 token 处理器转变为以 KV 缓存作为存储与通信介质的引擎集合。具体而言，它支持缓存卸载（跨查询的前缀复用）和预填充-解码（PD）拆分（跨引擎的缓存传输）。LMCache 的高性能与广泛应用源于以下贡献：(i) 经高度优化的 KV 缓存数据移动，包含批量数据移动操作、计算与 I/O 流水线等性能优化；(ii) 一个模块化的 KV 缓存连接组件，使 LMCache 与推理引擎的快速演化相解耦；(iii) 一流的控制 API，包括固定、查找、清理、移动和压缩等功能，支持在 GPU、CPU、存储及网络层之间灵活调度缓存。评估表明，将 LMCache 与 vLLM 结合使用，在多轮问答和文档分析等负载下，吞吐量最高可提升 15×。随着社区规模不断增长，LMCache 已广泛被企业级推理系统采纳，为未来 KV 缓存解决方案提供了宝贵经验。LMCache 的源代码位于：<https://github.com/LMCache/LMCache>。

1 引言

如今，大型语言模型（LLM）的推理增长速度已超过训练。LLM 推理支持广泛的应用，从交互式客户支持和代码生成，到基于检索的文档分析和 Agentic 工作流。然而，LLM 推理系统尚未跟上发展步伐——延迟（包括响应延迟和生成速度）和成本已成为瓶颈。

¹TensorMesh Inc., Foster City, CA, USA ²University of Chicago, Chicago, IL, USA. Correspondence to: Junchen Jiang <junchenj@uchicago.edu>.

*Equal contribution

当前大型语言模型推理系统的一个关键局限是，每个用户查询均由推理引擎的单个实例独立处理，且跨查询或推理引擎之间无法复用任何数据。这意味着，大型语言模型查询的生命周期，包括计算和输入/输出操作，均在单个推理引擎的 GPU 及其 GPU 内存中完成。每个查询以一个 token 序列作为输入，输出另一个 token 序列，一旦查询完成，推理过程中生成的所有输出或中间状态都会被丢弃。

这种设计尽管简单，却导致了冗余计算和资源利用率低下，业界提出了两项显著的解决方案来应对这些问题（图 1）。

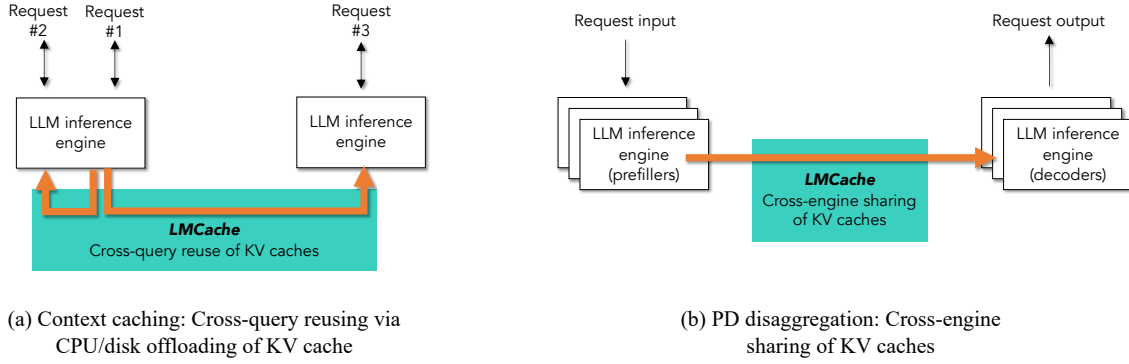


Figure 1. LMCACHE supports both context caching (KV cache offloading and sharing across queries) and PD disaggregation (cross-engine transfer of KV caches).

Cross-query caching to avoid redundant compute: 由于每个查询都是独立处理的, LLM 每次使用上下文 (前缀) 时都必须重新填充相同的上下文, 重复的预填充计算开销较大, 导致响应变慢, 因为用户在预填充完成之前无法看到第一个 token。然而, 如果我们将预填充前缀后 LLM 内部生成的 KV 缓存 在查询生命周期之外持续保存, 并在后续查询再次使用该前缀时重用此缓存 (例如相同的提示模板或检索到的文档片段), 就可以避免这种冗余计算。通过避免冗余计算, 跨查询上下文 (前缀) 缓存显著降低了首次标记时间 (TTFT) 以及预填充阶段的整体 GPU 资源消耗。

Prefill-decode disaggregation for higher utilization: 将预填充阶段 (计算密集型, 注重吞吐量) 与解码阶段 (内存密集型, 对延迟敏感) 共置于同一推理引擎中, 会导致资源利用率不足, 因为必须过度分配 GPU 以确保解码延迟始终较低, 且不会被预填充打断。为此, 业界尝试将注重吞吐量的预填充处理与对延迟敏感的对解码进行解耦。这种预填充-解码 (PD) 分离架构将所有查询的预填充阶段批量处理于一组推理节点上, 并将其 KV 缓存传输至另一组仅运行解码阶段的推理节点, 从而在高并发情况下缓解尾部解码延迟问题。

为了使这些理念具有实用性, 大语言模型推理系统必须引入新的键值缓存语义。特别是, 推理引擎应支持新接口, 能够提取常规推理调用中的键值缓存, 并按需重新加载到后续查询中。系统还必须

允许提取的键值缓存被持久化存储, 并在分布式推理引擎之间传输。最重要的是, 为了使此类接口扩展具有实用性, 键值缓存的提取、重载、存储和传输必须高效, 且新接口必须与快速演进的推理引擎 (如 vLLM (Kwon et al., 2023b) 和 SGLang (Zheng et al., 2024)) 保持兼容。

我们介绍了 LMCACHE, 这是首个开源库, 提供了这些新 KV 缓存语义的高性能实现。通过 LMCACHE, 可以高效地从推理引擎中提取和重新加载 KV 缓存, 将其存储在多层存储设备 (CPU 内存、本地磁盘、远程磁盘和 Redis) 中, 并在不同网络 (以太网、RDMA、NVLink) 之间传输。

LMCACHE 作出了三项不同的贡献。

#1. Highly optimized performance: LMCACHE 集成了多种性能优化技术, 使得在实际部署中存储和加载 KV 缓存既高效又可行。例如, LMCACHE 将操作批量处理, 以流水线方式执行 KV 缓存的存储与加载, 同时实现 GPU 计算与数据加载/存储的流水线并行 (如在当前层进行计算的同时加载下一层的 KV 缓存)。此外, LMCACHE 并非以推理引擎原生的小页大小为粒度进行 KV 缓存的存储/加载, 而是以可配置的块大小进行, 该块大小通常远大于页大小, 从而充分利用存储设备与 GPU 内存之间的带宽。LMCACHE 还通过实现零拷贝操作, 最大限度地减少了在不同存储层级之间移动 KV 缓存数据时的复制开销。

#2. Standardized interface with inference engines: LMCACHE 定义了标准化的连接器接口，这些接口与快速变化的推理引擎后端保持兼容。2025 年每周平均有 15-20 个新的开放权重模型发布，因此为了充分利用新硬件支持新模型，现代大语言模型推理引擎必须快速演进，这可能导致 GPU 内存中 KV 缓存布局的变化，从而影响 LMCACHE 接口。为解决这一问题，LMCACHE 设计并实现了一个模块化的 KV 缓存连接器接口，将 LMCACHE 与推理引擎后端解耦，从而使 LMCACHE 能够轻松适应推理引擎中不断演进的 API。

#3. Flexible KV cache management interface: LMCACHE 引入的接口增强暴露了 KV 缓存，这是大型语言模型推理中的一种新型数据结构。LMCACHE 暴露了 API，允许开发者和运维人员定位、移动、固定甚至压缩从推理引擎中提取的 KV 缓存。这些一级 API 使得上层应用（如查询调度器或路由器）能够做出更优决策，例如基于 KV 缓存的查询路由。

我们的评估表明，LMCACHE 在多种情景下始终优于开源推理框架中的内置 KV 缓存机制以及商业推理 API，其吞吐量最高提升 15 $\times$ ，延迟至少降低 2 $\times$ ，涵盖本地前缀缓存、分布式前缀重用以及 PD 分离等场景。除了量化性能提升外，LMCACHE 已被多家企业及开源项目采用，为生产环境中基于 KV 缓存的最优化提供了宝贵的实践经验。

本文余下部分详细介绍了动机 (§2)、LMCACHE 架构及关键设计选择 (§3-§6)、实验评估 (§8) 以及部署经验 (§9)。

2 动机

2.1 推理成本与延迟作为瓶颈

过去两年中，大模型在各类企业应用中的部署实现了爆炸性增长。随着基于大模型的应用程序用户数量和查询量的增加，推理的成本和延迟而非训练，成为了将大模型服务扩展至生产规模的主要瓶颈。

Inference cost: 除了大模型被越来越多的用户调用这一简单事实外，输入长度和输出长度的增加也导致了推理成本的上升，因为每个输入 token 和每个输出 token 都会带来额外的处理开销。推动输入和输出长度增长的四个趋势如下。(i) 每个用户与大模型的交互次数增多，形成了更长的用户历史记录，这些历史将作为上下文被前置到未来的模型输入中。(ii) 大模型越来越多地接收多模态输入，例如图像和视频，这些内容通常需要先变换为一长串 token 后才能输入到大模型中。(iii) 新一代大模型往往具有更长的上下文窗口，使得上下文/提示工程师可以简单地将更多 token 填入模型输入中。(iv) 最后，推理模型会产生异常长的输出，这些输出可能被用作工作流中后续查询的输入。

即使在 GPU 价格和性能持续提升的情况下，处理万亿 token 规模的工作负载仍可能导致每年数百万美元的计算支出 (Nie et al., 2024; Databricks Research, 2024)。查询之间的冗余计算（例如，重复计算相同的提示前缀数百次）会进一步加剧这些成本 (Zhang et al., 2024; Caylent, 2024)。

Inference delay: 在交互式应用中，尾部延迟（例如第 95/99 百分位）和中位延迟一样重要。最坏情况下的慢响应，通常由于资源争用或大上下文处理导致，会直接降低用户体验，甚至可能导致用户流失，这与视频流媒体卡顿影响观众体验的方式类似 (Liu et al., 2024a)。

特别是，行业报告和学术研究中的实证观察表明：

- 对于长上下文输入，尤其是当上下文长度超过 8000 至 16000 个 token 时，首次 token 生成时间 (TTFT) 主要受预填充阶段影响。
- 单流解码器的词元间延迟 (ITL) 保持较低，但当 GPU 计算资源在多个并发会话间共享时，该延迟会急剧上升。
- 尾部延迟受到严重影响，尤其是在需要大量提示重新计算或重新分配内存密集型资源的情况下。

2.2 KV 缓存作为一种跨领域最优化技术

KV 缓存最初被引入以通过将注意力状态以 K 和 V 张量的形式直接存储在 GPU 内存中，来加速单个查询的推理过程，这些状态分别对应输入 token 和先前生成的 token。KV 缓存有效地保存了在此查询中迄今为止已看到的每对 token 之间的注意力信息。简而言之，它是一种大语言模型原生的知识表示。

在所有流行的 Transformer 架构中，这些缓存仅在单个查询的生命周期内存储和使用，并在查询结束时被丢弃。然而，键值 (KV) 缓存可以扩展以支持生产级服务中的两项关键性能优化：

1. 上下文缓存 (*i.e.*, 跨查询 KV 缓存复用) 保持来自一个查询的 KV 缓存段，并在后续共享相同前缀的查询中重用它们。示例包括文档分析，其中相同的文档（块）在多个查询中保持不变，以及具有固定系统提示或长前言的多轮对话。前缀缓存减少了预填充阶段的冗余计算，直接降低了 TTFT 和每查询的 GPU 小时数 (Liu et al., 2024b; Gao et al., 2024; Jin et al., 2025b; Ren et al., 2025; Qin et al., 2025a; Jin et al., 2024; Chen et al., 2024; 2025)。
2. 预填充-解码 (PD) 分解 (即跨引擎的键值缓存传输) 将推理过程拆分为 *prefill* 阶段 (处理整个输入提示) 和 *decode* 阶段 (自回归生成 token)，并在不同的 GPU 或结点上执行。该方法通过最大化解码速度，避免预填充阶段的干扰，从而降低尾部延迟 (Zhong et al., 2024; Patel et al., 2024; Shi et al., 2025)。

Roles of KV caching: 两种最优化方法都关键地依赖于能够高效地在 GPU 内存、CPU DRAM、NVMe 存储或网络之间移动 KV 缓存段的能力。

- 上下文复用会将一个上下文的键值缓存 (KV cache) 保存尽可能长的时间，以便该上下文能够被重复使用，因此需要对键值缓存进行分层存储，并实现多个服务引擎实例之间高效共享键值缓存。

Message Size	Transfer Throughput
64KB	4GBps
256KB	13GBps
1MB	30GBps
10MB	46GBps
16MB	49GBps
100MB	49GBps

Table 1. 传输消息大小与使用 RCCL 传输库实现的传输吞吐量 (UCCL Team, 2025).

- PD 解耦需要在 GPU 之间快速传输 KV 缓存，通常通过 PCIe、NVLink 或 RDMA 实现，以最小化每个查询的端到端延迟，该查询跨越预填充结点和解码器结点运行。

2.3 高效键值缓存的挑战

尽管具有潜力，前缀缓存和 PD 解耦的实际应用仍受到三个相互关联的系统挑战的限制：

2.3.1 挑战 #1: 分页内存下的输入/输出低效

KV 缓存的存储与传输以往依赖于 PyTorch 序列化 (`torch.save` / `torch.load`) 或原始张量复制，典型的传输速度仅为 1GB/s 以下。这些方法在处理如 KV 缓存这类大型数据结构时引入了显著的延迟开销，并且缺乏与各种存储设备（本地或远程）的零拷贝支持，导致额外的 CPU-GPU 数据复制。

最近的高通量推理引擎，如 vLLM (Kwon et al., 2023b) 和 SGLang (Zheng et al., 2024)，使得 KV 缓存存储和传输更加具有挑战性。它们采用 分页注意力内存，将注意力缓冲区划分为小的、固定大小的页（通常为 16–64 KB）。例如，vLLM 在 Llama-3.1-8B-Instruct 模型中使用 62.5-KB 的页。分页内存架构被广泛使用，因为它能够提升批量处理能力和内存利用率。

然而，由于 KV 缓存的页并不总是连续的，分页内存架构显著增加了持久化或传输 KV 缓存所需的大量小尺寸 I/O 操作。传输如此小的数据块已知会导致网络带宽利用不足，并降低吞吐量 (Kwon

et al., 2025; NVIDIA Developer Forums, 2020; Meta Engineering, 2024)。先前的工作 (表 1) 表明, 在由八条 Broadcom Thor-2 400Gbps 网卡连接的两个 AMD GPU 结点的配置下, 传输大小必须达到至少 16 MB 才能使可用网络带宽饱和 (Zhou et al., 2025)。此外, 先前工作还表明, 仅当传输大小在兆字节范围 (例如 1–2MB) 时, 才能达到理论 PCIe 5.0 带宽的 75–80% (Xie et al., 2025)。

2.3.2 挑战 #2: 与快速演进的推理引擎兼容

随着人工智能的广泛应用, 大模型和硬件加速器正以极快的速度不断推出。到 2025 年, 平均每隔 4 天就会发布一款重要的大模型 (bes, 2025)。为此, 推理引擎也必须以同样的速度持续演进。

每次为适应新模型或硬件而更新时, 通常都会改变 GPU 内存分配, 进而影响 KV 缓存接口。例如, 当 vLLM 采用新的注意力核函数, 生成具有不同维度的 KV 缓存时, KV 缓存库必须随之更新, 以将新核函数输出的 KV 缓存格式转换为与 KV 缓存库兼容的格式。鉴于推理引擎的快速迭代, 持续应对这些频繁变化需要投入巨大的努力。

2.3.3 挑战 #3: 缺乏管理 API

随着键值缓存 (KV caching) 在大语言模型推理后端中成为核心组成部分, 各类组件 (除大语言模型推理引擎外) 以及机器学习运维团队都需要以感知 KV 缓存的方式做出决策。然而, 若缺乏统一的管理接口来定位、驱逐、固定或压缩缓存, 这些上层模块便无法做出明智的缓存放置或驱逐决策。这导致缓存利用率低下、存储重复以及不可预测的驱逐策略。例如, 推理查询路由器需要知晓 KV 缓存的位置, 以便将查询路由至已本地持有匹配前缀 token 的 KV 缓存的推理引擎实例 (如位于 CPU 内存中)。

此外, 应用程序现在也要求具备这样的 KV 缓存管理接口。例如, 2025 年初, 一家金融公司¹ 在生产环境中与 LMCACHE 紧密协作的团队提出了一种接口, 允许用户显式地将频繁访问的金融文档固定

¹为保密起见, 本报告中不披露企业用户名称。

在 KV 缓存系统中, 以更高效地访问热门上下文。作为另一个例子, 某智能体公司请求了一系列 API, 使他们能够识别给定内容的 KV 缓存、压缩 KV 缓存, 并在结点之间传输压缩后的 KV 缓存。

2.4 相关工作与现有解决方案

存在多种 KV 缓存处理机制, 但它们均未能完全解决上述挑战:

Inference frameworks: 自 2025 年 1 月发布 vLLM 生产栈 (vLLM project, 2025) 以来, 已出现多个开源分布式推理栈, 包括 Nvidia 的 Dynamo (NVIDIA Corporation, 2025)、AIBrix (Team et al., 2025)、llm-d (llm-d Project, 2025)、SGLang OME (Team, 2025) 以及 KServe (Contributors, 2025b)。它们专注于在 Kubernetes 上轻松部署推理引擎解决方案, 从技术角度看, 这些方案均支持基于负载或前缀缓存感知的多种查询路由器, 并支持 KV 缓存, 其中 vLLM 生产栈、Dynamo、llm-d 和 KServe 均采用 LMCACHE。

Inference engine-native KV caching: 开源推理引擎, 如 vLLM 和 SGLang, 同样支持原生的 GPU 到 CPU 的 KV 缓存传输, 但其设计仅针对单结点推理, 因此缺乏跨结点传输最优化或对 KV 缓存的分层存储支持。我们将在 §7 中评估它们的性能, 并与 LMCACHE 进行对比。

KV cache storage layers: 月饼 (Qin et al., 2025b)、Redis (Redis, 2025)、InfiniStore (ByteDance, 2025) 和 3FS (Contributors, 2025a) 提供了分布式对象存储或缓存功能, 但它们缺少推理引擎之间的高效“粘合”层来频繁跨不同存储层级高效移动小型张量, 或者与特定推理框架紧密耦合。

Proprietary implementations: 专有推理 API (如 Fireworks AI、Together AI) 内部实现了其自身的前缀缓存机制, 但这些功能与其闭源服务栈深度绑定, 对于部署自建基础设施的运营商而言并不可

用。

Source code for research: 一些研究提案已开源了其键值缓存优化的原型，包括前缀缓存 (Kwon et al., 2023a; Zheng et al., 2024; Yu et al., 2025; Gim et al., 2024; Ye et al., 2024; Lee et al., 2024; Zhao et al., 2024; Jin et al., 2024; Gao et al., 2024; Chen et al., 2025; Jin et al., 2025a; Yang et al., 2025)、PD 分离 (Zhong et al., 2024; Patel et al., 2024; Shi et al., 2025) 和键值缓存压缩 (Liu et al., 2024c; Jegou et al., 2024; Xiao et al., 2024a;b; Li et al., 2024; Qin et al., 2025c; Tang et al., 2024; Ge et al., 2024; Li et al., 2025; Du et al., 2025; Zhang et al., 2025)。然而，这些原型通常基于面向研究的推理框架（如 Huggingface Transformers）构建，尚未完全具备企业级可用性，或并未设计为能够随着 SGLang 和 vLLM 等快速发展的推理引擎生态系统同步演进。

3 LMCACHE 概述

LMCache 通过一个统一的高性能 KV 缓存层，解决了这些挑战，该层能够高效地存储、移动和显式管理分页内存推理引擎的 KV 缓存，使得前缀缓存和 PD 分离在企业级规模下成为可能。

作为 KV 缓存层，LMCACHE 位于大语言模型推理引擎与异构存储/网络设备之间（图 2）。其目标是为 KV 缓存的移动与管理提供一种标准化、高性能的基础架构，同时保持与 vLLM 和 SGLang 等快速发展的推理框架的兼容性。

3.1 架构

从高层次来看，LMCACHE 连接到推理引擎，以在 GPU 内存和存储后端之间移动 KV 缓存。图 3 展示了端到端系统。下面，我们通过两个示例工作进行说明：存储和检索 KV 缓存。

Store: 当新查询到达时，首先经过 KV 连接器，该组件准备元数据，例如分词后的输入提示以及相关页面的 GPU 内存地址。随后查询进入 token 处理

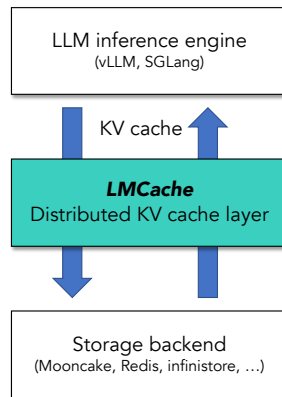


Figure 2. LMCACHE sits between LLM inference engines and heterogeneous storage/network devices.

器，该组件确定有多少新的 token 尚未存在于后端，需要进行存储。最后，存储管理器通过传输通道将这些新 token 的 KV 缓存保存到后端，该通道负责数据传输逻辑。

Retrieve: 当查询需要从后端加载 KV 缓存时，也会从 KV 连接器开始准备元数据。令牌处理器识别出后端中已存在的前缀匹配令牌数量。接着，事件管理器检查是否已见过相同的查询 ID。若是，则缓存的内存地址已被追踪，可直接返回至 GPU 连接器，由其将 KV 缓存重新加载回 GPU 内存。事件管理器还会启动如 §4.2 所述的异步、逐层加载事件。如果查询 ID 是新的，则将其转发给存储管理器，以查找存储的 KV 缓存对应的 CPU 内存地址。

Lookup: 当查询需要检查特定 token 的 KV 缓存是否存在于后端时，路由器等上层组件会查询缓存控制器。缓存控制器维护一个 token 池，用于记录当前存储在 KV 缓存后端的所有 token。每当一个 LMCACHE 实例存储或逐出 KV 缓存时，该实例内的 LMCACHE 工作线程会更新 token 池中的 token 状态。这确保了 token 池始终包含后端中 token 的最新信息。

3.2 关键部件

LMCACHE 由高性能的数据平面和灵活的控制平面组成。

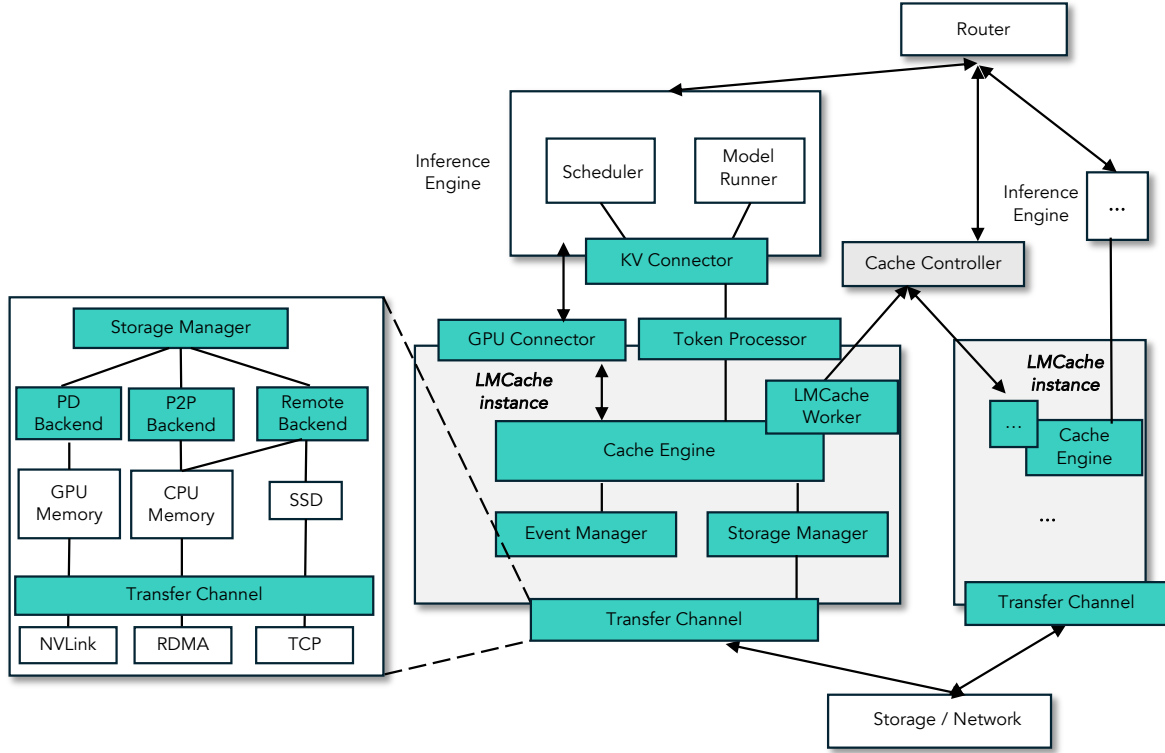


Figure 3. End-to-end system workflow for LMCACHE.

LMCache Worker (Data Plane): 每个推理引擎都与一个 LMCACHE 工作进程配对。LMCACHE 工作进程模块负责在 GPU 内存与其他层级或其它工作进程之间移动 KV 缓存。它实现了 KV 交换（到 CPU 或磁盘）以及 PD 分离（GPU-GPU 传输）。为了最大化吞吐量，工作进程采用内核优化的 GPU 缓冲区、异步分块 I/O 和逐层流水线技术，具体细节将在下一节中阐述。这些技术使得 LMCACHE 即使在处理小型分页内存对象（16-64KB）时也能维持接近 GPU 本地带宽的性能，这在 vLLM 和 SGLang 中是一种常见配置。

LMCache Controller (Control Plane): 控制器向系统操作员和上层调度器暴露了一个可编程的 API。它提供了缓存生命周期管理原语，例如固定 KV 段、压缩和解压缓存张量、在设备间迁移缓存，以及驱逐低优先级条目。控制器还维护一个虚拟化命名空间，实现了跨异构设备的缓存统一寻址。

LMCACHE 工作进程与控制器共同构成一个模块化系统，可无缝集成到企业级服务流水线中。工作进

程确保数据的快速传输，而控制器则提供高效资源利用所需的高层控制语义。

4 性能最优化

LMCACHE 的一个重要方面是提高跨设备的 KV 缓存移动效率。在企业级大模型推理中，LMCACHE 解决了三个关键挑战：

- 现代的大语言模型推理引擎以页为粒度管理键值缓存（KV cache）。²，这些小型单元通常为 20 KB-63 KB，适用于 Llama、Qwen、GPT-OSS 等流行模型。这种小单元在传输时效率较低，无法实现带宽饱和 (Xie et al., 2025; Zhou et al., 2025)；
- KV 缓存传输通常需要与大语言模型推理同时运行。这会带来两方面的开销。首先，如果数据传输与计算在同一个 CUDA 流中执行，数据移动可能会阻塞推理。其次，启动内存复制的 CUDA 函数会产生 CPU 开销，因为每次调用都会消耗

²每页在 vLLM 中为单层 16 个 token。

CPU 周期，当层数和页数较多时，这种消耗可能非常显著。

- 在大语言模型推理过程中，大量查询会产生大量的键值缓存 (KV caches)。将这些缓存复制到任何存储设备都会浪费空间并引入拷贝开销，从而降低推理速度。

这些挑战均源于开源和企业部署中的深刻教训。本节将详细描述这些挑战，并说明 LMCACHE 设计决策的动机。

4.1 批量操作

为解决由小的 KV 缓存单元引起的 I/O 低效问题，LMCACHE 引入了一组最优化。

Configurable Chunk Size: 相较于在页面层级传输 KV 缓存，LMCACHE 将来自多个层的多个页面分组为更大的块，默认大小为每个块 256 个 token³。这是通过一个中间的 GPU 缓冲区实现的。在存储时，多个页面（默认为 16 页）首先从 GPU 内存复制到缓冲区，然后集体卸载到低层级存储（例如，CPU 内存），并以数据块为粒度而非单个页面进行保存。在加载时，数据块首先从存储层检索到 GPU 缓冲区，随后被拆分为页面，再放入服务引擎的内部 GPU 内存中。LMCACHE 实现了一个定制化的 CUDA 内核以加速内存拷贝。

Batched Store/load Operations: LMCACHE 支持在多个存储设备之间存储和加载 KV 缓存，包括本地 CPU 内存、本地磁盘和远程磁盘。在实际应用中，KV 缓存通常需要并行地传输到不同的设备——例如，将热上下文的 KV 缓存存储在 CPU 内存中，同时将冷上下文卸载到本地磁盘。一种简单的顺序存储方法会低效利用可用带宽，因为向 CPU 内存传输时，磁盘带宽处于空闲状态。为解决此问题，LMCACHE 将不同层级间的存储和加载查询进行批量处理，从而聚合操作，并充分利用 GPU 与多个存储层级之间的带宽。

³块大小可配置以适应不同的 I/O 速度。

Delayed Decode KV Cache Storing: LMCACHE 还支持在解码过程中保存新生成的 KV 缓存。在一种简单的设计中，每个新生成的页会立即被卸载到低层级存储，导致频繁的小规模写入。相反，LMCACHE 会聚合多个页，并将它们作为一个块一起存储，从而提高 I/O 效率。

4.2 计算与输入/输出重叠

LMCACHE 的数据移动通常与大语言模型推理计算并行发生。例如，在解码过程中，键值 (KV) 缓存的计算与同时存储到存储设备中进行。简单的数据移动操作可能会阻塞大语言模型推理计算，反之亦然。LMCACHE 采用多种最优化技术，旨在将大语言模型推理计算与输入/输出 (I/O) 操作重叠。

Layer-wise pipelining: LMCACHE 通过逐层流水线实现 KV 缓存传输与推理计算的重叠。具体而言，它在每一层内为推理计算和数据移动分配分离的 CUDA 流。例如，在对第一层执行推理之前，其 KV 缓存会被加载到 GPU 缓冲区并变换为页。当第一层正在执行推理时，第二层的 KV 缓存被异步地加载到缓冲区，并同样进行变换。需要注意的是，第二层的 KV 缓存加载是在第一层的 KV 缓存被放入正确的分页内存之后才发生的。这种设计确保了仅需一个固定大小的 GPU 缓冲区——其大小相当于单一层的 KV 缓存——的同时，实现了数据传输与计算之间的重叠。

Asynchronous compute & prefetch: 在许多场景中，推理调度器接纳查询与查询的键值缓存实际用于推理之间存在时间间隔。例如，如果有 100 个查询同时到达，但推理引擎只能并行处理 50 个，则剩余的 50 个必须在队列中等待，直到开始推理。LMCACHE 利用这一空闲时间段，将排队查询的键值缓存从较慢的存储层级预取到较快的层级（例如，从远程磁盘预取到本地 CPU 内存）。因此，当实际推理计算开始时，所需的键值缓存可直接从更快的存储层级加载，显著减少加载延迟。LMCACHE 允许用户根据自身的延迟 SLO 和资源约束，配置预取的目标层级。

Process separation: 将 LMCACHE 的数据移动与推理引擎在单个进程中共享，由于 CPU 资源竞争，引入了 5%–10% 的延迟开销。为消除此开销，LMCACHE 将数据移动解耦至一个分离的进程，使其与推理计算过程隔离。此外，进程分离还支持多个推理引擎实例之间的内存共享。当数据移动与计算共存时，每个推理实例维护其自身的 CPU 内存池，因此实例之间无法访问彼此的 KV 缓存，除非通过进程间通信。相比之下，独立的数据移动进程管理一个统一的 CPU 内存池，允许多个推理实例从同一内存池中存储和加载 KV 缓存，从而提高效率并减少冗余。

4.3 极小数据复制

一种朴素的 KV 缓存移动实现会在每次传输步骤中创建额外的数据副本，尤其是在处理异构存储类型时，这会导致冗余的内存使用和不必要的开销。LMCACHE 通过仅维护最少必需的副本避免了这一问题。

Zero-Copy Operations: 在将键值缓存传输到远程存储或另一个 GPU 实例时（例如在分离式预填充场景中），LMCACHE 通过引用计数器最小化数据冗余。具体而言，当键值缓存被写入多个目标——如本地 CPU 内存、本地磁盘或远程磁盘——时，LMCACHE 会增加共享数据的引用计数器，而不是创建新的副本。每次写入完成，计数器减一，当计数为零时，数据被释放。这种设计确保了数据在并发写入操作之间共享，而无需不必要的复制，从而降低内存压力并提升效率。该技术类似于操作系统中的 PCB 计数器 (Strecker, 1978)。

Dynamic Offloading: 现代推理引擎（如 vLLM）在 GPU 内存中维护一个空闲页池，即 KV 缓存当前未被活跃查询使用的页。与将所有空闲页复制到 CPU 内存不同，LMCACHE 仅复制其中的一部分。该机制通过三个指针实现：

- **起始指针：** GPU 内存中空闲页区域的起始地址。
- **当前指针：** 已卸载到 CPU 内存的空闲页的索引。

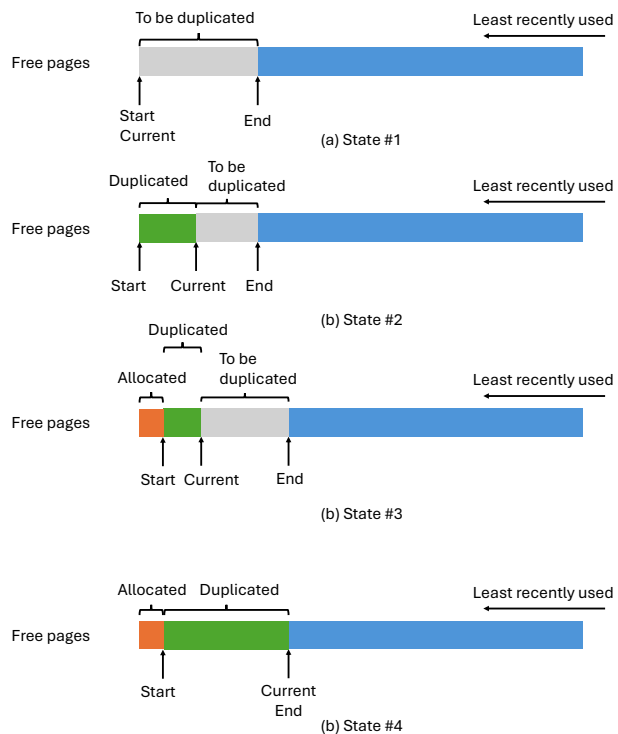


Figure 4. Illustration of dynamic offloading in LMCACHE.

- **结束指针：** 计划卸载的空闲页的结束地址。

如图 4 所示，动态卸载有四种可能的状态：

- **状态 #1 (初始化)：** 起始指针和当前指针重叠。起始/当前指针与结束指针之间的区域标记为待复制的页。
- **状态 #2 (进行中)：** 当前指针向结束指针移动。起始指针与当前指针之间的页面已全部卸载到 CPU 内存。
- **状态 #3 (查询到达)：** 当新的查询获取部分空闲页时，结束指针向前移动已分配页的数量。这确保了未来需要获取空闲页的活跃查询有足够的 GPU 内存可用。
- **状态 #4 (稳态)：** 当前指针与结束指针重叠，表示所有已调度的页面均已复制。

请注意，如果查询尝试分配超出当前指针范围的页，则分配必须暂停，直到当前指针向右移动足够远以覆盖所需的页。因此，该设计中的一个关键权衡在于：重复页的数量——即 GPU 与 CPU 内存之间

Function name	Description
<code>get_num_new_matched_tokens(query) → matched_tokens</code>	Returns the number of cache-hit tokens found in LMCACHE's backend.
<code>update_state_after_alloc(query, blocks, num_external_blocks)</code>	Updates whether a query needs to transfer KV cache from LMCACHE's backend.
<code>build_connector_meta(scheduler_output) → kv_connector_metadata</code>	Builds metadata for KV cache transfers between LMCACHE's backend and GPU memory, including GPU memory addresses for KV cache pages.
<code>start_load_kv(kv_pointers)</code>	Starts loading KV cache from lower-tier storage into GPU memory before LLM inference begins.
<code>wait_load_kv(kv_pointers, layer_id)</code>	Synchronizes on KV cache loading to ensure data is available when computation requires it.
<code>start_store_kv(kv_pointer)</code>	Starts offloading KV cache to lower-tier storage after computation.
<code>wait_store_kv(kv_pointer, layer_id)</code>	Synchronizes on KV cache storing to ensure the KV cache for the current layer is offloaded.

Table 2. LMCACHE 连接器中的函数。

由 `end pointer - start pointer` 定义的区域。较小的重复窗口虽然降低了重复率，但增加了分配阻塞的可能性。例如，如果仅重复一页，而一次推理查询需要三页，则查询必须等待当前指针和结束指针向前推进两页后才能继续。另一方面，若重复三页，则相同的查询可立即执行而无需阻塞，尽管这会带来更高的重复率。

5 连接 KV 缓存层与推理引擎的标准接口

现代大型语言模型推理引擎，如 vLLM 和 SGLang，正迅速发展以支持新发布的、架构多样的模型。例如，2025 年每周平均发布 15–20 个新模型。支持这些新架构通常需要对推理引擎进行非平凡的修改，例如增加对滑动窗口注意力（Sliding Window Attention）或多头潜在注意力（Multi-Head Latent Attention）的支持。这些代码更改经常改变内部 KV 缓存的管理方式，使得 LMCACHE 无法以临时应对的方式进行适应。

为应对这一挑战，LMCACHE 引入了一种标准化的 KV 缓存连接器接口，将 KV 缓存管理与推理引擎

后端解耦。该设计确保了无论潜在推理引擎如何演进，LMCACHE 均能保持兼容性。

具体而言，在 vLLM 中，LMCACHE 通过两个关键接口进行集成：1) 调度器，其中预填充 token 的数量直接影响调度决策，并由 LMCACHE 改变（即如果 LMCACHE 中发生缓存命中，则需要新预填充的 token 数量将发生变化）；以及 2) 模型执行器，推理计算在此处进行，且在执行前后必须完成 KV 缓存的加载与存储。

本节余下部分列出了表 2 中的所有接口，讨论了重要 API 的设计，并追踪查询如何端到端地与这些接口交互。

表 2 中列出的接口构成了 LMCACHE 在低层级存储中进行 KV 缓存加载和存储的基础。其中，前三个接口在 vLLM 调度器内实现，它们根据 LMCACHE 的 KV 缓存后端中匹配到的 token 数量来准备必要的元数据。其余四个接口位于模型运行器中，负责在推理引擎与 LMCACHE 的 KV 缓存后端之间执行实际的 KV 缓存传输。

综合起来，当查询到来时，调度器首先调用 `get_num_new_matched_tokens`，该函数向 LM-CACHE 查询后端的缓存命中 token。随后，`update_state_after_alloc` 函数根据 LM-CACHE 提供的匹配 token 信息，决定 vLLM 中每个页面是否需要从外部存储后端加载。如果缓存命中 token 数量大于零，则调用 `build_connector_meta` 函数，以准备必要的元数据，用于从存储设备加载或存储 KV 缓存。

当查询到达模型运行器后，在分层流水线的情况下，`start_load_kv` 被调用以开始将第一层的 KV 缓存加载到 GPU 内存中。然后在每一层的 LLM 推理计算开始之前，调用 `wait_load_kv` 以同步该层的 KV 缓存加载，并启动下一层的 KV 缓存加载。在每一层推理计算完成后，在分层情况下，调用 `wait_store_kv` 等待前一层的 KV 缓存存储完成，然后调用 `start_store_kv` 以开始存储新生成的 KV 缓存层。

在非分层流水线的情况下，在第一层的 LLM 推理开始之前，会调用 `start_load_kv` 以阻塞方式将整个 KV 缓存加载到 GPU 内存中。只有在 KV 缓存被放置到正确的 GPU 内存分页地址后，才会进行 LLM 推理。然后，在当前调度迭代的 LLM 推理完成后，会调用 `start_store_kv` 同步地将生成的 KV 缓存存储到低层级存储中。

6 控制器接口

许多场景中，上层应用需要查看 KV 缓存的位置并对之进行操作。例如，查询路由可以从知道前缀缓存命中 token 最多的区域中获益。为了支持此类用例，LM-CACHE 提供了如表 3 所示的 API，用于查询和操作 KV 缓存，例如定位缓存条目或在实例间迁移它们。

LM-CACHE 中的 KV 缓存控制器由两个组件组成：一个集中式管理器和每个实例的工作者。传入的控制器查询由集中式管理器处理，该管理器将适当的操作分发给运行在每个 LM-CACHE 实例中的工作者。本节其余部分通过几个示例应用说明如何使用

控制器接口。

KV cache-aware routing: 在这种情况下，高层级查询路由器希望将查询定向至前缀缓存命中数最多的实例。路由器首先调用 `lookup(tokens)`，由集中管理器将查询请求分发给各个 LM-CACHE 实例，以识别包含匹配前缀 token 的实例，随后调用 `query_ip(instance_ids)` 将实例标识映射为 IP 地址。最终，查询将被转发至命中数最高的 IP 地址。

KV cache migration: 当持有 KV 缓存的实例发生故障或需要负载均衡时，可能需要将 KV 缓存跨实例迁移。`move(source, destination, tokens)` 接口将 tokens 对应的 KV 缓存从源实例移动到目标实例。

KV cache clearance: 应用程序在切换模型或回收内存时可以清除缓存。`clear(tokens, instance, location)` 函数会移除指定 instance 内存存储在特定存储位置 location（如 GPU 内存或 CPU 内存）的 tokens 对应的 KV 缓存。

Pinning KV cache in GPU memory: 某些上下文，例如聊天机器人应用中的系统提示，应保留在 GPU 内存中以实现快速访问。`pin(instance, location, tokens)` 函数将指定的 KV 缓存固定在给定实例的选定存储层级（GPU 内存）中。

用户还可以调用其他更高级的接口来操作 KV 缓存，例如 `compress(tokens, instance, location, compression_method)` 函数，该函数会使用 `compression_method` 对 LM-CACHE 实例中特定 location 处存储的 KV 缓存进行压缩，从而减少其在存储中的占用空间。用户可根据自身需求自由调用这些接口，显式管理应用程序中的 KV 缓存。

7 评估

7.1 设置

我们将在三种不同场景下评估 LM-CACHE，如表 4 所示。这三种场景是 LM-CACHE 用户常用的典型配

LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference

Interface	Description
lookup(tokens) $\rightarrow$ {instance_id: hit_tokens}	Returns the instance names of KV caches containing prefix-match tokens with the given token list.
query_ip(instance_ids) $\rightarrow$ IP	Returns the IP addresses for the corresponding instance_ids
move(source, destination, tokens)	Moves the KV cache of the given token list from a source instance to destination.
clear(tokens, instance_id, storage_device)	Clears the KV cache for corresponding tokens from the storage_device in instance .
pin(tokens, instance, storage_device)	Pins the KV cache for corresponding tokens at the storage_device in instance .
compress(tokens, instance, storage_device, compression_method)	Compresses the KV cache for the corresponding tokens with a compression method and store it at the storage_device in instance .

Table 3. LMCACHE 连接器中的函数。

Scenario Acronym	Single-node / Multi-node	Network Medium	Real-world Examples	Hardware:
CPU Offload	Single-node	N.A.	Single-node CPU Offloading	对于单节点评估，我们在 GMI Cloud (GMI Cloud, 2025) 提供的 8× H100 服务器上运行 LMCACHE。由于不同模型需要不同数量的 GPU 来提供服务，我们在评估中分配能够成功启动每个模型所需的极小点 H100 GPU 数量。对于多节点评估，我们使用与单节点设置相同的 GPU 数量，并配置一个远程存储后端，利用 CPU 内存进行 KV 缓存存储。对于 PD 分离架构，预填充器和解码器实例均配置与单节点评估相同的 GPU 数量，且预填充器与解码器实例通过 NVLink 连接。
Central Storage	Single-node	Ethernet	Centralized Storage Server	
PD	Single-node	NVLink	PD Disaggregation	

Table 4. 评估场景设置。

置。

Models: 我们将 LMCACHE 与基准方案在工业界广泛采用的流行开源模型上进行比较: meta-llama/Llama-3.1-8B-Instruct、meta-llama/Llama-3.1-70B-Instruct、Qwen/Qwen2.5-Coder-32B-Instruct、Qwen/Qwen3-Coder-480B-A35B-Instruct-FP8、Qwen/Qwen2.5-72B-Instruct。

Datasets: LMCACHE 在多个数据集上进行了评估,包括模拟的多轮问答系统、来自 LongBench (Bai et al., 2024) 的长上下文问答系统,以及来自 vLLM 官方基准测试脚本 (Kwon et al., 2023c) 的随机数据集。

Metrics: 每个实验中，我们展示了首个 token 的生成时间 (TTFT)，即预填充延迟，以及 token 间延迟 (ITL)，即两个连续输出 token 生成之间的平均延迟。对于按组件分析 (将 CPU 卸载或 PD 分离的延迟进行分解)，我们分别报告每个组件的延迟。

Baselines: 我们将 LMCACHE 与多个基准方法进行比较，包括 1) vLLM 配合前缀缓存，该方法实现了前缀缓存，但仅将有限数量的 KV 缓存保留在 GPU 内存中；2) 商业解决方案 A、B 和 C，它们提供专用端点服务，为用户预留 GPU 以运行用户自定义的模型。

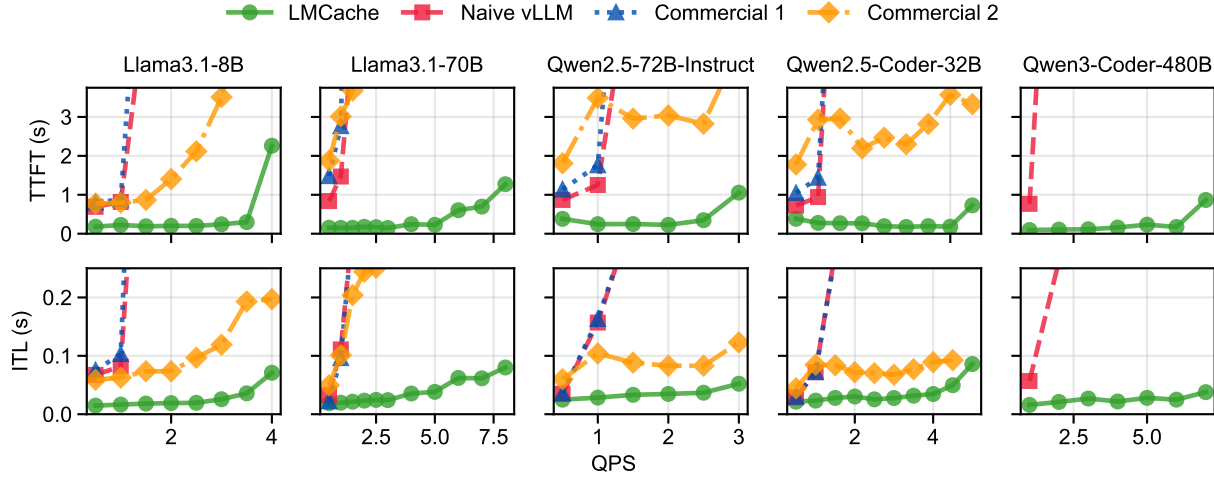


Figure 5. 单结点评估结果。

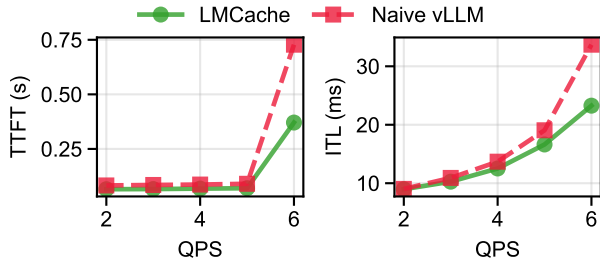


Figure 6. 真实迹评估结果。

7.2 单结点 CPU 卸载

我们首先在表 4 所示的 CPU 卸载场景中评估 LMCACHE。在此实验中，我们使用多轮问答工作负载，模拟典型的基于聊天机器人的文档分析场景。默认情况下，每个大语言模型查询包含 10K token，由一个文档（约 12 页 PDF）作为上下文和一个独特的简短问题组成。由于较小的模型通常能处理更多且更长的查询，因此 Llama-3.1-8B-Instruct 模型最多可接收 20K token 输入。大语言模型输出的最大长度为 100 token。聊天会话初始有 40 个用户，随后根据指定的到达率（QPS）陆续加入更多用户。我们将 LMCACHE 可卸载到 CPU 内存的最大 KV 缓存设置为 500 GB。

如图 5 所示，LMCACHE 在 TTFT 和 ITL 两个指

标上均持续优于所有对比方法。例如，在低 QPS 情况下（如 $QPS = 1$ ），LMCACHE 在五个评估模型中，相较于最强的基准方法，在相同 TTFT 下实现了 2.3–14 $\times$ 更高的查询处理速率（即吞吐量）。在 ITL 方面，LMCACHE 同样优于基线方法，因为后者在生成首个 token 前存在较长延迟，进而导致后续 token 的生成被排队等待。

对于 Qwen3-Coder-480B，商业选项 #1 和 #2 均不提供对该模型的部署支持。

Understanding LMCACHE’s gains: LMCACHE 在多个方面优于基准方法。与仅将 KV 数据缓存于 GPU 内存的朴素 vLLM 预填充相比，LMCACHE 利用了 CPU 侧卸载技术。由于 CPU 内存可存储的 KV 缓存远多于 GPU 内存，LMCACHE 实现了显著更高的缓存命中率。通过其高效的 CPU 到 GPU 数据加载机制，缓存数据能够快速调取，从而加速推理过程。我们对闭源商业方案的对比是在黑盒模式下进行的，因为它们的内部实现并未公开。从端到端的结果来看，我们推测商业选项 #1 缺乏将 KV 缓存卸载至二级存储的机制。相比之下，商业选项 #2 可能支持将 KV 缓存卸载至二级存储，但其性能仍不及 LMCACHE。

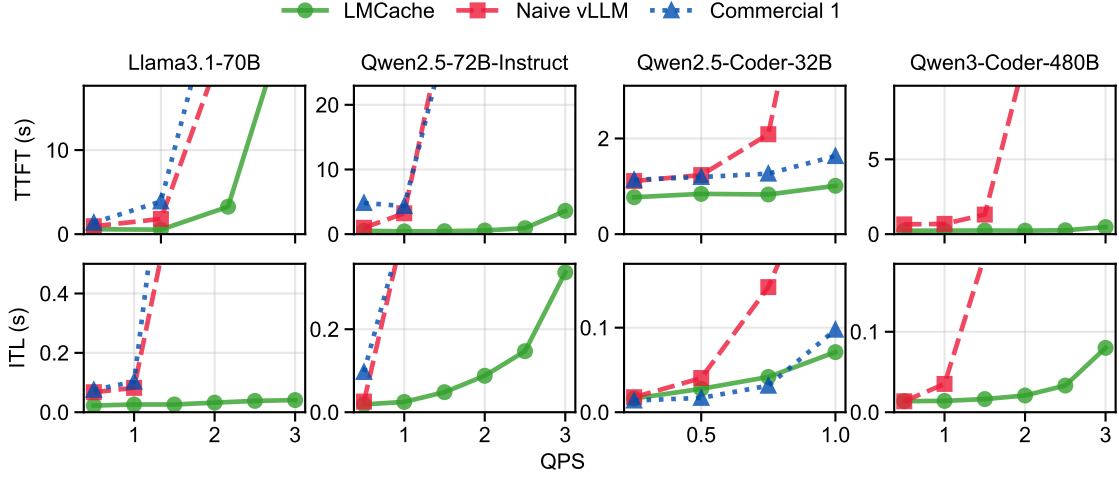


Figure 7. 远程后端评估结果。

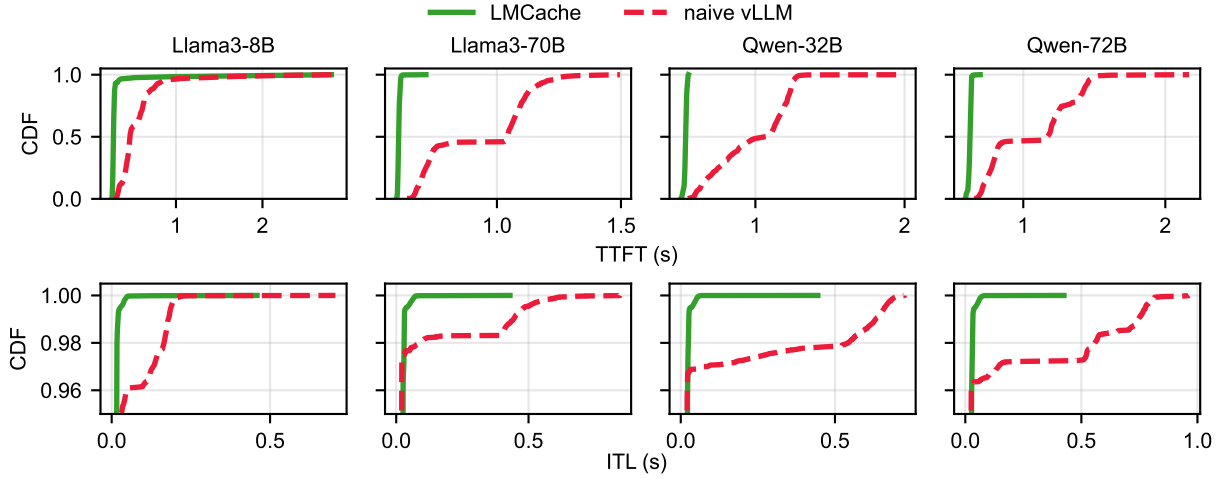


Figure 8. PD 的 TTFT 与 ITL CDF 对比。

7.3 集中式存储服务器

接下来，我们通过一个连接到 GPU 实例（带宽为 15 Gbps）的集中式远程服务器运行 LMCACHE 实现 KV 缓存共享，其设置遵循表 4 中的中心化存储方案。在本次实验中，我们使用 LongBench (Bai et al., 2024) 提供的 TriviaQA 数据集进行评估，该数据集是长上下文评估中广泛采用的基准。我们遵循 vLLM 官方基准测试脚本 (Kwon et al., 2023c)，根据指定的 QPS 按泊松分布生成推理查询。

如图 7 所示，LMCACHE 在不同 QPS 水平下始终优

于所有基准方法，在推理吞吐量上提升了 1.3-3 $\times$ 。这一提升源于远程后端能够存储远多于 CPU 内存的 KV 缓存，从而实现了更高的缓存命中率。

然而，我们注意到，从远程后端加载 KV 缓存的延迟高于从 CPU 内存加载，因为远程后端的带宽要低得多。因此，加载延迟甚至可能超过预填充延迟，尤其是在输入上下文较短或模型较小时，因为在这种情况下预填充速度非常快——我们将在后续 7.7 中展示这一情形。

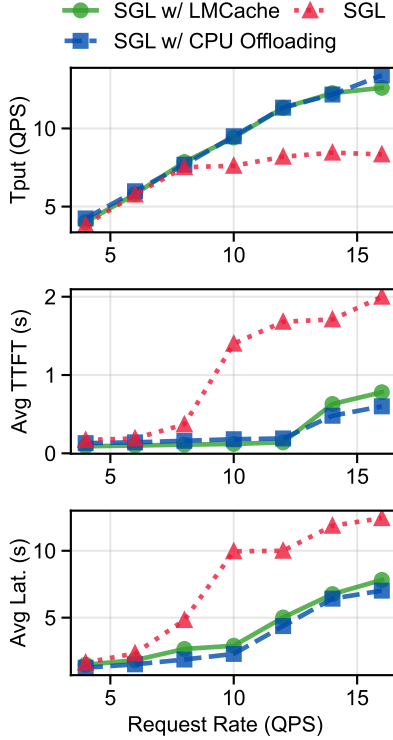


Figure 9. 与 SGLang 的 CPU 卸载结果相比。

7.4 PD 解聚

在本实验中，我们评估了在 PD 分离情景下的性能表现。在此情景下，我们使用官方基准测试脚本对随机输入和输出工作负载下的 LMCACHE 与 vLLM 的原生 PD 分离方案进行对比。输入长度为 8K token，输出长度为 200 token。如图 8 所示，展示了 LMCACHE 与 vLLM 原生 PD 分离方案的 95 百分位 TTFT，表明 LMCACHE 在尾部延迟方面显著更优。在平均 TTFT 方面，LMCACHE 同样大幅优于 vLLM 原生 PD 分离方案。具体而言，LMCACHE 在四个模型上将平均 TTFT 降低了 1.53–1.84 $\times$ ，将平均 ITL 降低了 1.12–1.66 $\times$ 。

LMCACHE 相比基准方法的性能提升源于其对 PD 分解更高效的设计。具体而言，LMCACHE 将分块预填充过程中生成的每个 KV 缓存块复制到预填充器实例的 GPU 内存缓冲区中，然后将其传输到解码器实例对应的缓冲区。接收到后，KV 缓存被复制到解码器实例的分页内存中。

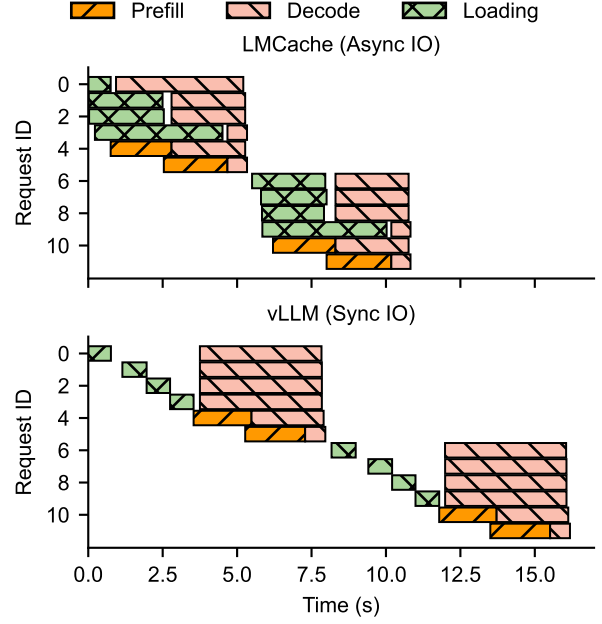


Figure 10. 通过请求异步化，LMCACHE 将 KV 缓存加载与推理计算（预填充或解码）重叠进行。

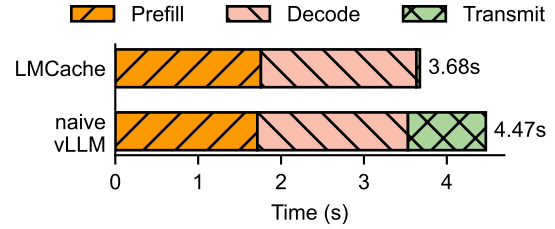


Figure 11. PD 的延迟分解。

相比之下，vLLM 的原生 PD 分离将预填充器生成的分页 KV 缓存直接发送到解码器，使用 NIXL 的内存拷贝功能。该功能以预填充器侧 KV 缓存页的内存地址作为输入，并将其复制到解码器侧的目标地址。然而，当 KV 缓存的分页内存分散在预填充器的 GPU 内存中时，传输以页为单位进行，如 §4 所讨论，这会导致带宽利用率低下。

7.5 真实迹驱动的评价

我们还在公司 F 提供的真实生产环境迹线 (trace) 上评估了 LMCache。⁴ 该迹包括查询到达时间戳和实际的多轮对话输入。值得注意的是,同一用户会话中的后续轮次通常会重用早期轮次的对话历史,导致前缀重复使用。平均而言,该迹中的输入长度约为 4000 个 token。

由于我们无法访问公司 F 的专有模型,因此使用 Sao10K/L3-8B-Lunaris-v1 模型重放迹线。为了使实验易处理,我们将原本持续数天的迹线拉长,并应用随机采样,使得我们运行的工作负载能在一小时内完成。

如图 6 所示,在不同 QPS 的真实情景下,LMCache 始终优于朴素的 vLLM,显著降低了 TTFT 和 ITL,展现了其在真实场景中的有效性。值得注意的是,当 QPS 从 2 降至 5 时,LMCache 仍比朴素的 vLLM 提升了 25%,而当 QPS 为 6 时,LMCache 相较于朴素的 vLLM 提升了 49%。

7.6 逐组件评估

为了进一步理解 LMCache 带来的收益,我们还进行了组件级分析,以分解端到端系统中每个组件的延迟。

PD disaggregation: 图 11 展示了大语言模型推理的延迟分解,包括预填充和解码计算,以及预填充器与解码器实例之间 KV 缓存的传输。预填充和解码计算时间在 LMCache 与 vLLM 的原生 PD 分离架构中是相同的。然而,如 §7.4 所讨论, vLLM 的原生设计以更细粒度传输 KV 缓存,导致带宽利用率低下。相比之下,LMCache 采用更高效的 KV 缓存传输机制,实现了显著更快的传输速度,从而降低了 PD 分离架构中的整体端到端延迟。

CPU offloading: 表 7.6 中,我们进行了消融实验,以测试 LMCache 与 vLLM 原生 CPU 卸载从 CPU 获得的加载带宽。LMCache 实现的传输带宽

⁴因保密协议而匿名。

Method	Achieved Bandwidth
LMCache	400 Gbps
vLLM's Native CPU Offloading	88 Gbps

Table 5. LMCache 在从 CPU 内存加载 KV 缓存时,相比 vLLM 的原生 CPU 交换,实现了更高的加载带宽。

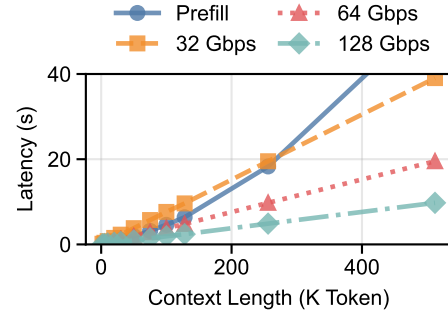


Figure 12. 不同上下文长度的影响。

高于 vLLM 原生 CPU 卸载的原因在于传输粒度。当原生 CPU 卸载按页进行数据移动时,LMCache 则按块进行数据传输。每次传输操作都会触发一次 CUDA 内存复制,这需要预先准备元数据,并在之后发送完成信号。这些每次传输的操作为每个内存复制内核增加了开销。通过每次复制传输更大的数据块,LMCache 降低了整体开销,从而实现了更高的有效带宽。

Asynchronous Compute: 我们还展示了 LM-Cache 的异步计算在降低端到端延迟方面的优势。图 10 展示了查询加载和推理计算的时间线。该图从一次更长运行的中间部分绘制,以方便说明。如图所示,若不采用查询异步处理,预填充/解码计算与加载会依次进行。而通过查询异步处理,预填充/解码计算可以与 KV 缓存加载重叠,从而将端到端延迟减少 1.46 $\times$ 。

7.7 敏感性研究

我们还进行了多项敏感性评估,以观察在不同上下文长度和不同类型的远程后端下,LMCache 的延迟如何变化。

Impact of context lengths: 图 12 展示了在 B200 机器上的预填充延迟以及不同网络带宽下的缓存加载延迟。当网络带宽较低 (*i.e.*, 32 Gbps) 时, 只有当输入上下文长度超过 256K 个 token 时, LMCACHE 的 KV 缓存加载才优于朴素预填充。相比之下, 当带宽较高 (*i.e.*, 64 或 128 Gbps) 时, LMCACHE 的加载在所有上下文长度下均能保持更低的延迟, 优于朴素预填充。这些结果表明, LMCACHE 的 KV 缓存加载应具备自适应能力: 在低带宽情况下, 仅当上下文长度超过加载速度优于预填充的交叉点时, 才应启用加载。

7.8 SGLang 结果

尽管我们的主要评估使用了 vLLM, 我们也评估了集成 SGLang 的 LMCACHE。图 9 展示了在两块 H100 GPU (TP=2) 上运行 Qwen3-32B 时, 启用 LMCACHE 的 CPU 卸载功能的结果。与未启用 CPU 卸载的 SGLang 相比, LMCACHE 实现了更高的吞吐量以及更低的平均 TTFT 和平均端到端延迟。与 SGLang 原生的 CPU 卸载相比, LMCACHE 实现了相当的性能表现。这些结果证实了 LMCACHE 在另一款推理引擎上同样有效。尽管 SGLang 原生的 CPU 卸载在 SGLang 上实现了与 LMCACHE CPU 卸载相当的性能, 但它缺乏一个分布式存储后端, 无法高效地将数据跨分层存储设备 (如本地磁盘和远程 CPU/磁盘资源) 进行卸载。

8 现实世界的教训与经验

8.1 缩放 KV 缓存存储与卸载

LMCACHE 在生产系统中采用的一个明显趋势是, 需要存储的 KV 缓存越来越多, 显然超出了 GPU 内存所能容纳的数量。早期的部署将所有 KV 缓存保留在 GPU 内存中, 但长上下文和多用户工作负载可能很快耗尽 GPU 内存。因此, 现在许多公司需要在必要时将 KV 数据卸载到 CPU RAM、CPU 内存池甚至磁盘中。这可以避免淘汰有用的缓存条目——若不进行卸载, 后续查询将触发相同输入上下文的完整预填充, 通常涉及长上下文, 这会显著

延长 TTFT。

有趣的是, 一些团队 (包括公司 R) 已经尝试使用远程存储设备。远程数据获取的速度比 GPU 内存访问慢好几个数量级, 但实际测试表明, 它仍然能够带来加速和成本节约。通过网络检索之前计算好的 KV 缓存, 可能比在已加载的 GPU 上重新计算更快, 这得益于 LMCACHE 的 KV 缓存加载优化 (§4)。尽管从远程加载 KV 缓存的开销较大, 但可以与推理计算流水线并行处理, 例如在预填充尚未命中缓存的请求时, 或在解码计算期间进行。这些经验表明, 即使速度较慢且成本较低的存储, 也可以被高效地用于存储和加载 KV 缓存。

另一种在生产环境中使用的技术是将 CPU 卸载与 PD 解耦相结合。在公司 T, 预填充端将键值缓存不仅发送到解码器端, 还卸载到预填充实例的 CPU 内存中, 从而同时实现 CPU 卸载和 PD 解耦的优势。

8.2 新兴应用场景与意外结果

虽然聊天机器人是 KV 缓存复用最早显而易见的有益应用场景之一, 但新的使用场景正在迅速涌现。最近, 公司 L 和公司 M 的大规模推荐系统也采用了大模型作为推荐模型, 成为 KV 缓存复用的重要应用场景。在这些系统中, 大模型被用作嵌入模型来预填充上下文, 不生成 token, 并且会被同一用户的多个查询频繁复用。此外, 这些上下文通常非常长。缓存这些 KV 缓存可以直接消除昂贵的预填充阶段, 因此是 LMCACHE 的理想应用场景。

尽管多轮对话长期以来一直是 KV 缓存复用的正则应用场景, 但实际部署中揭示了若干重要洞察。许多系统最初采用滑动上下文窗口, 以在有限的 GPU 内存中容纳更多 token 进行推理。然而, 这种方法常常因丢弃有用的历史信息而降低生成质量, 并降低前缀缓存命中率, 因为截断后的上下文已不再是前缀。公司 T 和 F 的部署经验表明, 保留完整的对话历史并将其 KV 缓存卸载到更大的内存层级, 是一种更有效的做法。通过避免截断, 系统在共享前缀上实现了显著更高的缓存命中率, 从而降低了每

个请求的 GPU 内存使用量。此外，在生产环境中，企业观察到前缀缓存命中率远高于预期。

Company B 的采用经验表明，行业用户已接受有损最优化，例如 KV 缓存压缩，以获得更好的系统性能。尤其是在开放式聊天机器人应用中，由于不存在单一的“正确”答案，使用压缩的 KV 缓存（例如量化 KV 缓存）可能不会损害用户体验，但可显著降低内存使用量和 I/O 时间。即使在金融公司中，团队也发现，若整体系统吞吐量得到提升，使用压缩 KV 缓存带来的微小差异也是可以接受的。

8.3 集成与部署挑战

在生产环境中的大语言模型推理系统中部署缓存层存在其自身的实际挑战和经验教训。我们采用 LM-CACHE 时发现的一个关键经验是，许多公司在将缓存层部署到生产环境时更倾向于使用容器化环境（如 Docker 镜像），因为他们通常不会深入到 LM-CACHE 的源代码中。

在大语言模型推理的缓存层中，另一个重要方面是容错性和透明性。由于缓存层位于推理引擎之下，应对外部用户保持不可见，因此鲁棒性至关重要。许多公司强调，如果缓存系统发生故障或性能下降，不应导致模型服务中断。例如，LMCACHE 引入了容错的键值（KV）缓存读取机制：若读取过程中出现错误，LMCACHE 会报告哪些部分的 KV 缓存成功读取，确保推理引擎不会因 KV 缓存读取失败而崩溃，并保证生成结果的正确性。

我们还观察到，对模型服务引擎与缓存引擎之间实现清晰分离的需求日益增长。许多存储公司，例如公司 I、R、C 和 W，已明确要求将 KV 缓存管理与核心推理代码解耦，因为他们希望尽可能少地触及核心推理逻辑。尽管目前 LMCACHE 尚未解决此问题，但趋势已经十分明确：大语言模型推理引擎正与多种外部 KV 缓存管理服务相配合。

最后，我们发现确保 LMCACHE 与新兴推理框架之间的兼容性比预期更容易。LMCACHE 最初是在 vLLM 基础上设计和开发的。然而，只要其他框架采用分页注意力机制，将 LMCACHE 与它们集成

也十分简便，例如 SGLang。展望未来，我们预计 LMCACHE 将同样容易集成到基于分页注意力的其他推理框架中。

8.4 采纳、经验教训与未来展望

我们的旅程始于 2024 年夏季的一个学术原型。当时，KV 缓存仍是一个小众概念，主要出现在少数研究论文中。但到了 2025 年春季，局势已发生改变——KV 缓存成为工业界提升大语言模型推理效率的关键技术。LMCACHE 的采用在 2025 年初急剧增长，众多公司开始合作并投入使用。

有趣的是，尽管 LMCACHE 最初只是一个学术原型，但我们发现其工业界用户远多于学术界用户。一个可能的原因是缺乏一套易于使用的可编程接口。作为一个具体例子，大量研究文献探讨了从 KV 缓存中丢弃 token 以将其压缩为更小的张量。然而，在 LMCACHE 中实现 token 丢弃并不简单，因为 LMCACHE 假设输入和输出到 LMCACHE 的 token 数量应保持一致。此外，注意力模块中的任何中间状态对 LMCACHE 都是隐藏的，因此需要访问或修改注意力状态的研究工作很难在 LMCACHE 上实现。

最后，我们学成，用于高性能计算的编程语言并不总是构建大语言模型推理基础设施的最佳选择。早期，一些组织（例如公司 R 和 I）认为，为了实现高效率，KV 缓存管理必须用 Rust 或 C++ 来实现。相比之下，我们使用 Python 开发了 LMCACHE，这反而成为更优的选择。Python 不仅简化了与现有推理框架的集成，还降低了社区贡献的门槛，从而实现了快速的功能开发和迭代。当然，LMCACHE 内部某些高性能数据加载模块仍需要依赖基于 CUDA 的实现。

9 结论与展望

本文提出了 LMCACHE，这是首个开源且被广泛采用的面向企业级大语言模型推理的生产就绪型 KV 缓存层。通过将 KV 缓存视为一类首等数据结构，而非推理过程中的内部副产品，LMCACHE 将大语言模型引擎

从孤立的 token 处理单元转变为一个分布式计算与存储生态系统。在多种工作负载和模型上的评估表明,相较于开源基准与商业推理 API,LMCACHE 均能持续带来显著的吞吐量提升和延迟降低。除了性能优势外,LMCACHE 已在生产环境中迅速获得采纳,企业利用其 CPU 任务卸载、分层存储以及 PD 资源解耦能力,在万亿 token 规模部署中保持低延迟并降低成本。真实场景的部署还揭示了新的应用机会,例如推荐系统中的 KV 缓存复用以及开放式聊天机器人中的有损压缩,充分展现了 LMCACHE 在不同应用领域中的多功能性。

展望未来,LMCACHE 指向了一种更广泛的转变:**人工智能原生数据(如 KV 缓存)将越来越多地成为缩放大语言模型推理和智能体工作负载的基础架构**。通过将 KV 缓存确立为标准化的存储与通信媒介,LMCACHE 为未来系统奠定了基础,使推理不再被视为孤立的会话,而是作为持久化、缓存感知的计算网络。我们希望本文提出的架构设计、最优化方法以及部署经验,能够为下一代大语言模型基础设施提供指导,使得人工智能原生数据(如 KV 缓存)不仅是一种最优化手段,更是实现高效、可靠且可扩展推理的核心基本构件。

LMCACHE 的源代码位于: <https://github.com/LMCache/LMCache>。

10 致谢

我们感谢 LMCACHE 社区提供的宝贵支持与贡献,包括毛宝龙和郑春晓负责管理远程连接器, Martin Hickey 负责 GitHub 基础设施,张怀政、Siddhant Ray、顾卓涵和李晗辰负责编写和维护文档,张睿负责 LMCACHE 的部署,张启正和 Hussain Mohammad 提供了富有见地的反馈。

参考文献

- Best 44 large language models (llms) in 2025. <http://explodingtopics.com/blog/list-of-llms>, 2025. Accessed: 2025-09-18.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. URL <https://arxiv.org/abs/2308.14508>.
- ByteDance. InfiniStore: Kv cache store for distributed llm inference. <https://github.com/bytedance/InfiniStore>, 2025. Accessed: 2025-09-10.
- Caylent. Prompt caching: Saving time and money in llm applications. <https://caylent.com/blog/prompt-caching-saving-time-and-money-in-llm-applications>, 2024. Accessed: 2025-09-18.
- Shiyang Chen, Rain Jiang, Dezhi Yu, Jinlai Xu, Mengyuan Chao, Fanlong Meng, Chenyu Jiang, Wei Xu, and Hang Liu. Kvdirect: Distributed disaggregated llm inference, 2024. URL <https://arxiv.org/abs/2501.14743>.
- Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. IMPRESS: An Importance-Informed Multi-Tier prefix KV storage system for large language model inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 187–201, Santa Clara, CA, February 2025. USENIX Association. ISBN 978-1-939133-45-8. URL <https://www.usenix.org/conference/fast25/presentation/chen-weijian-impress>.

- DeepSeek AI Contributors. deepseek-ai/3fs: A high-performance distributed file system for ai training and inference workloads. <https://github.com/deepseek-ai/3FS>, 2025a. GitHub repository, MIT License.
- KServe Contributors. kserve/kserve: Standardized distributed generative and predictive ai inference platform for scalable, multi-framework deployment on kubernetes. <https://github.com/kserve/kserve>, 2025b. GitHub repository, Apache-2.0 license.
- Databricks Research. How long should you train your language model? <https://www.databricks.com/blog/how-long-should-you-train-your-language-model>, 2024. Accessed: 2025-09-18.
- Dayou Du, Shijie Cao, Jianyi Cheng, Luo Mai, Ting Cao, and Mao Yang. Bitdecoding: Unlocking tensor cores for long-context llms with low-bit kv cache, 2025. URL <https://arxiv.org/abs/2503.18773>.
- Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with cachedattention, 2024. URL <https://arxiv.org/abs/2403.19708>.
- Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. Model tells you what to discard: Adaptive kv cache compression for llms, 2024. URL <https://arxiv.org/abs/2310.01801>.
- In Gim, Guojun Chen, Seung-Seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In Phillip B. Gibbons, Genady Pekhimenko, and Christopher De Sa, editors, *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*. mlsys.org, 2024. URL https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66ca1703fe34705a4368c3014c1966-Abstract-Conference.html.
- GMI Cloud. Gmi cloud: Gpu cloud solutions for scalable ai & inference. <https://www.gmicloud.ai/>, 2025. Provides high-performance GPU infrastructure and services for AI training, inference, and deployment. Founded in 2023, based in Mountain View, CA. Retrieved September 15, 2025.
- Simon Jegou, Maximilian Jeblick, Alessio Devoto, Jiwei Liu, and David Austin. Kvpress: Efficient kv cache compression for long-context llms, 2024. URL <https://github.com/NVIDIA/kvpress>. Version 1.2.0.
- Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. Ragcache: Efficient knowledge caching for retrieval-augmented generation, 2024. URL <https://arxiv.org/abs/2404.12457>.
- Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Zhuoqing Mao. Compute or load KV cache? why not both? In *Forty-second International Conference on Machine Learning*, 2025a. URL <https://openreview.net/forum?id=W0y0ta06lQ>.
- Shuowei Jin, Xueshen Liu, Qingzhao Zhang, and Zhuoqing Mao. Compute or load KV cache? why not both? In *Forty-second International Conference on Machine Learning*, 2025b. URL <https://openreview.net/forum?id=W0y0ta06lQ>.
- Wook Kwon et al. Demystifying nccl: An in-depth analysis of gpu-based collective communication.

- arXiv preprint arXiv:2507.04786*, 2025. URL <https://arxiv.org/abs/2507.04786>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626, New York, NY, USA, 2023a. Association for Computing Machinery. ISBN 9798400702297. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023b. URL <https://arxiv.org/abs/2309.06180>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023c. Software: vLLM. <https://github.com/vllm-project/vllm/tree/main/benchmarks>.
- Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172, Santa Clara, CA, July 2024. USENIX Association. ISBN 978-1-939133-40-3. URL <https://www.usenix.org/conference/osdi24/presentation/lee>.
- Junyan Li, Yang Zhang, Muhammad Yusuf Hassan, Talha Chafekar, Tianle Cai, Zhile Ren, Pengsheng Guo, Foroozan Karimzadeh, Colorado Reed, Chong Wang, and Chuang Gan. Commvq: Commutative vector quantization for kv cache compression, 2025. URL <https://arxiv.org/abs/2506.18879>.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation, 2024. URL <https://arxiv.org/abs/2404.14469>.
- Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services, 2024a. URL <https://arxiv.org/abs/2404.16283>.
- Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. Cachegen: Kv cache compression and streaming for fast large language model serving, 2024b. URL <https://arxiv.org/abs/2310.07240>.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache. *arXiv preprint arXiv:2402.02750*, 2024c.
- llm-d Project. llm-d: A kubernetes-native high-performance distributed llm inference framework. <https://github.com/llm-d/llm-d>, 2025. Accessed: 2025-09-10.
- Meta Engineering. Roce networks for distributed ai training at scale. <https://engineering.fb.com>

- /2024/08/05/data-center-engineering/roce-network-distributed-ai-training-at-scale/, Aug 2024. Accessed: 2025-09-18.
- Chengyi Nie, Rodrigo Fonseca, and Zhenhua Liu. Aladdin: Joint placement and scaling for slow-aware llm serving, 2024. URL <https://arxiv.org/abs/2405.06856>.
- NVIDIA Corporation. Nvidia dynamo: A datacenter-scale distributed inference serving framework. <https://github.com/ai-dynamo/dynamo>, 2025. Accessed: 2025-09-10.
- NVIDIA Developer Forums. Why is the transfer throughput low when transferring small size data (gpu host/device transfers). <https://forums.developer.nvidia.com/t/why-is-the-transfer-throughput-low-when-transferring-small-size-data-from-host-to-device-or-device-to-host/153962>, 2020. Accessed: 2025-09-18.
- Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Riccardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting, 2024. URL <https://arxiv.org/abs/2311.18677>.
- Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation — a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, Santa Clara, CA, February 2025a. USENIX Association. ISBN 978-1-939133-45-8. URL <https://www.usenix.org/conference/fast25/presentation/qin>.
- Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A kvcache-centric disaggregated architecture for llm serving, 2025b. URL <https://arxiv.org/abs/2407.00079>.
- Ziran Qin, Yuchen Cao, Mingbao Lin, Wen Hu, Shixuan Fan, Ke Cheng, Weiyao Lin, and Jianguo Li. Cake: Cascading and adaptive kv cache eviction with layer preferences, 2025c. URL <https://arxiv.org/abs/2503.12491>.
- Redis. Redis enterprise software reference —redis documentation. <https://redis.io/docs/latest/operate/rs/references/>, 2025. Accessed: 2025-09-10.
- Zebin Ren, Krijn Doekemeijer, Tiziano De Matteis, Christian Pinto, Radu Stoica, and Animesh Trivedi. An i/o characterizing study of offloading llm models and kv caches to nvme ssd. In *Proceedings of the 5th Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems, CHEOPS '25*, page 23–33, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400715297. doi: 10.1145/3719330.3721230. URL <https://doi.org/10.1145/3719330.3721230>.
- Xiaoxiang Shi, Colin Cai, Junjia Du, and Zhihao Jia. Nexus: proactive intra-gpu disaggregation of prefill and decode in llm serving, 2025. URL <https://arxiv.org/abs/2507.06608>.
- William D. Strecker. Vax-11/780: A virtual address extension to the dec pdp-11 family. In *Proceedings of the National Computer Conference*, pages 967–980, Montvale, NJ, 1978. AFIPS Press.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context llm inference, 2024. URL <https://arxiv.org/abs/2406.10774>.
- The AIBrix Team, Jiaxin Shan, Varun Gupta, Le Xu, Haiyang Shi, Jingyuan Zhang, Ning Wang,

- Linhui Xu, Rong Kang, Tongping Liu, Yifei Zhang, Yiqing Zhu, Shuwei Jin, Gangmuk Lim, Binbin Chen, Zuzhi Chen, Xiao Liu, Xin Chen, Kante Yin, Chak-Pong Chung, Chenyu Jiang, Yicheng Lu, Jianjun Chen, Caixue Lin, Wu Xiang, Rui Shi, and Liguang Xie. Aibrix: Towards scalable, cost-effective large language model inference infrastructure, 2025. URL <https://arxiv.org/abs/2504.03648>.
- The SGLang Team. Ome: Revolutionizing llm infrastructure with model-driven architecture. <https://lmsys.org/blog/2025-07-08-ome/>, July 2025. Blog post, LMSYS Org.
- UCCL Team. Everything you want to know about kv cache transfer engine. <https://uccl-project.github.io/posts/kv-transfer-engine/>, August 2025. Blog post, August 13, 2025.
- vLLM project. vllm production stack: Reference system for k8s-native cluster-wide deployment with community-driven performance optimization. <https://github.com/vllm-project/production-stack>, 2025. Version vllm-stack-0.1.7, released Sep 3, 2025.
- Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context llm inference with retrieval and streaming heads, 2024a. URL <https://arxiv.org/abs/2410.10819>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2024b. URL <https://arxiv.org/abs/2309.17453>.
- Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving, 2025. URL <https://arxiv.org/abs/2508.18572>.
- Huan Yang, Renji Zhang, Mingzhe Huang, Weijun Wang, Yin Tang, Yuanchun Li, Yunxin Liu, and Deyu Zhang. Kvshare: An llm service system with efficient and effective multi-tenant kv cache reuse, 2025. URL <https://arxiv.org/abs/2503.16525>.
- Lu Ye, Ze Tao, Yong Huang, and Yang Li. ChunkAttention: Efficient self-attention with prefix-aware KV cache and two-phase partition. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 11608–11620, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.623. URL <https://aclanthology.org/2024.acl-long.623/>.
- Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys ’25, page 144–158, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400711961. doi: 10.1145/3689031.3696086. URL <https://doi.org/10.1145/3689031.3696086>.
- Hailin Zhang, Xiaodong Ji, Yilin Chen, Fangcheng Fu, Xupeng Miao, Xiaonan Nie, Weipeng Chen, and Bin Cui. Pqcache: Product quantization-based kvcache for long context llm inference, 2025. URL <https://arxiv.org/abs/2407.12820>.
- Yan Zhang, Fei Li, Yanzhao Tang, Bingsheng He, Xiaoyong Wu, and Jeffrey Xu Yu. Optimizing llm queries in relational workloads. *arXiv preprint*

arXiv:2403.05821, 2024. URL <https://arxiv.org/abs/2403.05821>.

Yilong Zhao, Shuo Yang, Kan Zhu, Lianmin Zheng, Baris Kasikci, Yang Zhou, Jiarong Xing, and Ion Stoica. Blendserve: Optimizing offline inference for auto-regressive large models with resource-aware batching, 2024. URL <https://arxiv.org/abs/2411.16102>.

Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. URL <https://arxiv.org/abs/2401.09670>.

Yang Zhou, Zhongjie Chen, Ziming Mao, Chon-Lam Lao, Shuo Yang, Pravein Govindan Kannan, Jiaqi Gao, Yilong Zhao, Yongji Wu, Kaichao You, et al. An extensible software transport layer for gpu networking. *arXiv preprint arXiv:2504.17307*, 2025.